

《人工智能基础》大作业

一、选题背景和需求

选题一：AI Agent 框架搭建——从“会说话”到“能做事”

自 2023 年大语言模型（LLM）爆发以来，AI 的定位正从“聊天机器人”向“智能代理（Agent）”快速演进。虽然 LLM 具备强大的常识和推理能力，但它本质上是一个受限于训练数据的闭环系统——它无法感知实时天气、无法进行精确的数学运算，也无法操作你的本地文件。AI Agent 的出现打破了这层“次元壁”，通过引入 ReAct（推理+行动）等范式，赋予了模型调用外部 API 和自主决策的能力，使其成为了人工智能通识教育中最前沿、最具工程价值的方向。AI Agent 框架搭建背景大语言模型（LLM）虽然能聊天和写文章，但本身不会上网、不会算数、不会操作文件——它只是一个“文字接龙”的模型。AI Agent 的核心思想是：让 LLM 学会“调用工具”和“自主决策”，从一个“会说话的模型”变成一个“能做事的智能体”。比如你问它“帮我查一下今天北京天气并写一首相关的诗”，Agent 会自己决定先调用天气 API、再把结果交给 LLM 写诗。这是 2023 年以来 AI 应用层最重要的范式之一，也是 ChatGPT、Claude 这些产品背后的关键技术。**作业目标**从零搭建一个简化版的 AI Agent 框架，让它能够：接收用户的自然语言指令 → 自主选择合适的工具 → 调用工具并获取结果 → 把结果整理后回复用户。整个过程支持多轮“思考-行动”循环。

核心技术要点：

1. Prompt 工程：设计一个能让 LLM 输出“工具调用指令”的系统提示词，参考 ReAct 范式（Reasoning + Acting，让模型交替输出“思考”和“动作”）
2. 解析与执行循环：解析 LLM 输出的 JSON 或特定格式，找到要调用的工具和参数，执行后把结果再喂回给 LLM，直到任务完成
3. 对话历史管理：维护一个 messages 列表，处理上下文长度限制

备注：推荐技术栈 Python + OpenAI / DeepSeek / 通义千问 等任意一家的 API（学生可以申请免费额度，或者用本地部署的开源小模型如 Qwen2.5-3B）。**不**

允许直接用 LangChain、LlamaIndex 这类高层框架

要求：

1. 至少实现 3 个工具（如计算器、维基百科搜索、本地文件读写）
2. 能完成一个需要多步调用的任务（比如“查一下爱因斯坦的出生年份，然后算一下他活了多少岁”）
3. 提交代码 + 一个 demo 视频或对话截图

进阶（加分项）：

支持流式输出、加上短期记忆 / 长期记忆、做一个 Web 界面（Gradio/Streamlit）、实现多 Agent 协作（一个规划 Agent + 一个执行 Agent）。学到什么理解 LLM 应用层最核心的设计模式，建立“提示词即程序”的思维，熟悉 API 调用和 JSON 解析这些工程基本功。

选题二：NumPy 实现神经网络——揭开自动微分的“黑盒”

在 PyTorch 和 TensorFlow 等工业级框架盛行的今天，开发者只需一行 `loss.backward()` 即可完成复杂的反向传播。这种便利性虽然提高了生产力，却也导致许多学习者陷入了“调包侠”的困境：对矩阵维度变换一脸茫然，对梯度消失和梯度爆炸缺乏直觉。只有亲手用 NumPy 编写过每一个神经元、推导过每一行求导公式，才能真正理解深度学习是如何从数百万次微小的参数修正中“进化”出智能的。这个题目的目标是亲手把神经网络的每一个零件造出来，理解前向传播、反向传播、梯度下降这些深度学习最核心的机制。这是从“会用 AI”到“懂 AI”的分水岭。作业目标只用 NumPy（不允许使用 PyTorch、TensorFlow 等任何自动微分框架），从零实现一个多层感知机（MLP），完成在 MNIST 手写数字数据集上的训练和推理，准确率达到 95% 以上。

核心技术要点：

1. 前向传播：实现全连接层、激活函数（ReLU、Sigmoid、Softmax）、损失函数（交叉熵、MSE）的矩阵化计算

2. 反向传播：手动推导每一层的梯度公式，按链式法则实现反向传播。这是这个题最硬核的部分，需要学生在纸上把矩阵求导算清楚

3. 优化器：实现 SGD，进阶可实现 Momentum、Adam

4. 训练循环：mini-batch 划分、shuffle、训练集/验证集/测试集评估、loss 和 accuracy 曲线绘制

工程细节：参数初始化（Xavier / He 初始化）、学习率调整、过拟合时加 L2 正则或 Dropout 推荐技术栈 Python + NumPy + Matplotlib（画训练曲线）。MNIST 数据集可以从官网或 torchvision.datasets 下载（仅用来读数据，不用它的模型）。

要求：

1. 一个能跑通的训练脚本，MNIST 测试集准确率 $\geq 95\%$
2. 一份实验报告，包含反向传播的数学推导（手写拍照或 LaTeX 都行）、训练曲线、超参数对比实验（学习率、批大小、网络深度的影响）
3. 代码结构要清晰，把 Layer、Loss、Optimizer 拆成独立的类（模仿 PyTorch 的 API 风格）

进阶（加分项）：

实现卷积层和池化层做成 CNN（准确率冲到 99%）、实现 BatchNorm、用自己的框架做一个 Fashion-MNIST 或 CIFAR-10 的实验、写一个简化版的 autograd 自动求导引擎（参考 Karpathy 的 micrograd）。学到什么彻底搞懂神经网络的数学原理和工程实现，以后再看 PyTorch 源码或读论文不会发怵。这是整个深度学习生涯里最值得投入时间的一次造轮子。

选题三：下棋 AI 与博弈搜索——经典 AI 的智慧与魅力

在深度学习统治视野之前，人工智能的代名词是“搜索”。从深蓝击败卡斯帕罗夫到 AlphaGo 的早期原型，博弈树搜索算法始终是决策类 AI 的核心。与依赖大数据驱动的深度学习不同，经典博弈算法体现的是一种“符号智能”和

“理性逻辑”。通过在巨大的状态空间中进行高效的 Minimax 搜索 与 Alpha-Beta 剪枝，我们能看到 AI 如何在没有“经验”的情况下，通过纯粹的算力与逻辑推演展现出令人惊叹的博弈水准。这个题目让同学体会：AI 不止有大模型，“聪明地搜索”本身就是一种强大的智能。作业目标实现一个能和对战的棋类 AI，棋种可在井字棋、五子棋（建议 8×8 或 9×9 小棋盘）、黑白棋（Othello）中任选。AI 要能在合理时间内（比如每步 ≤ 5 秒）走出有水平的棋。

核心技术要点：

1. 游戏状态表示：用二维数组表示棋盘，实现合法走法生成、胜负判断、终局检测
2. Minimax 算法：构建博弈树，假设对手也最优，递归找当前最好的走法
3. Alpha-Beta 剪枝：在 Minimax 基础上裁掉不可能影响结果的分支，搜索效率能提升一个数量级以上
4. 评估函数：当搜索深度受限不能搜到终局时，需要一个启发式函数评估“当前局面对我多有利”。这是体现同学“棋感”和工程能力的地方，比如五子棋可以数活三、活四、冲四等棋型
5. 迭代加深搜索：在时限内逐步加深搜索深度，保证总能给出一个合理走法

备注：推荐技术栈 Python（开发快、好调试）或 C++（性能好，能搜更深）。界面可以用 pygame 做图形界面，或者纯命令行打印棋盘。**不允许调用现成的棋类 AI 库。**

要求：

1. 一个能正常对战的 AI（人 vs AI），有基本的图形界面或命令行交互
2. 实现 Minimax + Alpha-Beta 剪枝
3. 实验报告对比：纯 Minimax vs Alpha-Beta 在不同搜索深度下的耗时和胜率；不同评估函数对棋力的影响

进阶（加分项）：

实现置换表（Transposition Table，缓存已搜索过的局面）、走法排序优化剪枝效率、用 MCTS（蒙特卡洛树搜索）替代 Minimax 做一版（这是 AlphaGo 的核心算法之一）、做一个 AI vs AI 的天梯赛系统让全班的 AI 互相切磋。学到什么掌握递归与回溯的工程化运用、理解状态空间搜索这一 AI 经典范式、训练算法优化和性能调试的能力。完成后会对“AI 不等于深度学习”有更全面的认识。

二、开题需求和调研

在正式进入代码开发之前，请各小组深入调研选题的技术背景，并明确团队分工与执行计划。**以下部分是开题报告需要完成的。每组交一份开题文档，提交在学在浙大上。截止时间为 5 月 24 日**

2.1 技术背景调研与文献分析

请各小组针对所选题目，通过阅读技术博客、学术论文或开源项目文档，回答以下问题：

技术现状：当前该领域的主流解决方案是什么？（例如：选题一可调研 ReAct 范式与 OpenAI Function Calling 的区别；选题二可调研不同激活函数的优劣；选题三可调研当前主流的评估函数设计思路）

核心原理预研：简述你对本题目核心逻辑的理解（如：Agent 如何通过 Prompt 约束输出格式？BP 算法中链式法则如何体现？Alpha-Beta 剪枝为何能减少搜索量？）

2.2 团队分工细节

请明确每位成员的具体职责。根据自己小组选的课题，明确每一位同学在课题中的角色和负责的部分。（例如）

姓名	角色定位	具体负责模块（示例）
XXX（组长）	架构设计与集成	负责整体框架搭建、核心算法逻辑实现（如 ReAct 循环/搜索算法）。
XXX	算子/工具开发	负责功能性开发（如 Agent 工具函数编写 / 神经网络算子推导 / 棋盘 UI 开发）。
XXX	实验测试与文档	负责超参数调优、数据采集、Demo 视频录制及最终实验报告撰写。

2.3 任务进度表

请根据各选题的进度节点，细化各小组的执行计划。（至少需求 2 周以上工作量）例如（这里只给一个大方向的参考，同学需根据自己的课题详细规划每一周的工作任务）：

第一周：环境准备与原理验证

第二周：核心功能开发

第三周：优化与实验对比

第四周：结项与交付

三、大作业交付

每个小组需提交一个压缩包，命名为 [选题编号]-组长姓名-学号，包含以下内容。截止时间 6 月 18 日

3.1 源代码

3.2 实验报告

实验报告必须包含以下内容：（可以在开题报告的基础完善）

核心原理与算法设计（例如选题一：展示你的 ReAct 提示词模版，绘制 Agent 的决策闭环流程图。选题二：给出前向传播、损失函数以及反向传播的梯度求导公式，给出数据在各层流转时的 Matrix Shape 变换过程。选题三：描述

Minimax 递归的终止条件，详细列出你的分值权重表)

工程实现细节 (例如：模块化说明，简述各个类 (Class) 或函数的功能。

和一些关键的数据处理。描述对应选题实现的具体细节。)

实验结果与分析 (例如选题一：截取一段 Agent 成功进行“多步推理”的对话日志，并分析其思考过程。选题二：必须包含 Loss 曲线 和 Accuracy 曲线。选题三：必须对比 纯 Minimax vs Alpha-Beta 剪枝 在同一搜索深度下的耗时/搜索节点数。)

3.3 演示视频

3.4 (如果参加答辩) 答辩 PPT

四、答辩和评分标准

4.1 评分标准

我们将根据以下权重进行评分：

- 1. 线下答辩展示(10%)**
- 2. 功能完整性(35%)**：是否达到了选题的要求。(每一个选题下有对应的要求)
- 3. 实验报告(35%)**：实验报告的内容格式是否完整规范。
- 4. 演示视频和 PPT(5%)**：根据同学提交的演示视频和 PPT 具体打分
- 5. 进阶挑战(15%)**：鼓励尝试进阶方向 (如：Gradio 界面、多 Agent 协作、MCTS 算法等)

4.2 答辩

每组 8 分钟，5 分钟展示，3 分钟提问。**自选进行线下答辩 (6 月 11 日)**，不需要介绍选题背景，考虑时间问题，同学们 PPT 直接介绍选题中关键的核心原理和算法实现。